

smallgameagent 实验报告

基于 LLM/VLM 与规则的小游戏 Agent

smallgameagent 项目组

2026 年 7 月 18 日

目录

1	smallgameagent 实验报告（修正与扩充版）	2
1.1	0. 一页结论	2
1.2	1. 环境打通	3
1.3	2. 在线 gameplay 基线与多 Agent 矩阵	3
1.4	3. 空间、时间、策略与效率问题（本轮观察）	3
1.5	4. 本地 VLM 基准（RTX 5060 Laptop 8GB）	4
1.6	5. 云端 API 混合实验	4
1.6.1	5.1 连通性与延迟	4
1.6.2	5.2 视觉 struct 提取（JSON schema）	4
1.7	6. Agent 通信与记忆	5
1.8	7. 训练准备	5
1.9	8. 测试与代码质量	6
1.10	9. 后续建议（按优先级）	6

1 smallgameagent 实验报告 (修正与扩充版)

> 2026-07-18 (本轮)。配套: __CODE_0000__ (方案)、__CODE_0001__ (过程数据)。

> 本轮重点: 修正报告中的不实数据, 补充 RTX 5060 本地 VLM、云端 API 混合、Agent 通信/记忆的实测结果。

1.1 0. 一页结论

- __BOLD_0002__: WSL2 下使用 Playwright 绑定的 Chromium + __CODE_0000__ 可正常初始化 Cocos 场景; 之前 headless swiftshader 报 __CODE_0001__ 的阻塞已解决。
- __BOLD_0001__: 在 SSD_00461P01 (塔防) 上跑了 rule / multi / multi-bus / multi-bus-memory 四种模式, 各 30 步。当前 __BOLD_0002__, 根因不是 Agent 通信或记忆, 而是该游戏的塔防机制 (放置/升级塔、自动战斗) 未被现有的 __CODE_0000__ 摇杆驱动覆盖; Agent 能移动但无法完成游戏目标。
- __BOLD_0003__: 显式总线 __CODE_0000__ 相比纯 __CODE_0001__ 墙钟增加约 6% (30 步 28.4s vs 26.7s), 每轮产生 15 条总线消息; __CODE_0002__ 文件型策略记忆可正常写入/读取, 但未在本游戏产生可量化的分数提升 (因动作空间不匹配)。
- __BOLD_0000__: llama.cpp CUDA12 后端 + 4-bit 权重 + q4_0 KV-cache 量化可加载 Qwen3.5-4B/9B 与 gemma-4-E4B-it。视觉 struct 提取: __BOLD_0001__; Qwen3.5-9B 2/3 解析成功, ~45 tok/s, ~16.9s/帧; Qwen3.5-4B 0/3 解析成功 (思考链占满输出或 markdown fence 后带多余文本)。
- __BOLD_0000__: OpenCodeGo 余额已可调用。mimo-v2.5 / kimi-k2.7-code / kimi-k2.6 文本/视觉请求均成功。结构化视觉提取: __BOLD_0001__ (22-38s/帧); __BOLD_0002__ (8-10s/帧), 原因是在当前 1024 token 预算内先输出大量思考过程, JSON 被截断。
- __BOLD_0000__: QLoRA 训练脚本与 processed-runs 数据转换已就绪; ssh5090 仍不可访问, 尚未开始微调。
- __BOLD_0004__: 本轮修复 verifier/orchestrator/critic 的 None/Verdict 类型处理、报告生成脚本的 ruff 问题; __CODE_0000__ 671 passed / 6 failed (失败与缺失的 __CODE_0001__ 目录及 __CODE_0002__ 扫描根目录副作用有关), __CODE_0003__ 全绿。

1.2 1. 环境打通

项	结论
浏览器	Playwright 绑定 Chromium (___CODE_0000___), WSLg 显示可用; 必须加 ___CODE_0001___
云端 LLM	___CODE_0000___; kimi 模型不能带显式 ___CODE_0001___, 客户端已按模型名自动省略
本地 VLM	直接用 LM Studio 捆绑的 ___CODE_0000___, GGUF 放在 ___CODE_0001___, 与 LM Stud
游戏	SSD_00461P01 塔防已能加载并返回 probe 状态 (___CODE_0000___, player=___CODE_000

1.3 2. 在线 gameplay 基线与多 Agent 矩阵

实验脚本: ___CODE_0000___, 单游戏 SSD_00461P01, 30 步/模式。

模式	步数	composite	activity	progress	stall 步	墙钟	主要决策来源
rule	30	0.000	0.000	0.000	29	26.7s	___CODE_0000___
multi	30	0.000	0.000	0.000	29	26.8s	___CODE_0000___
multi-bus	30	0.000	0.000	0.000	29	28.4s	___CODE_0000___ (30 次)
multi-bus-memory	30	0.000	0.000	0.000	29	28.4s	___CODE_0000___ (30 次)

关键发现:

- Agent 能发出移动指令(___CODE_0000___), 但 rubric 的 ___CODE_0001___/___CODE_0002___ 为 0, 说明 ___BOLD_0003___——Hero 在基地附近小范围移动, 未击杀敌人、未解锁塔、未推进波次。
- ___CODE_0000___ 只给出 ___CODE_0001___, 而该游戏的实际交互是 ___BOLD_0002___ 或 ___BOLD_0003___, 不是走到 UnlockItem 上。
- 因此当前最高杠杆不是“多 Agent 通信”或“记忆”, 而是 ___BOLD_0000___。

___CODE_0000___ 额外数据: 每轮 Observer→StateMapper→DecisionAnalyst→Verifier→Critic—共 ___BOLD_0001___ (observe×2、perceive×2、decide×4、verify×4、critic×2、memory×1)。Verifier 触发 2 轮 critic 后仍放行, 未造成显著延迟。

1.4 3. 空间、时间、策略与效率问题（本轮观察）

- ___BOLD_0003___: probe 能稳定返回节点世界/屏幕坐标; 当前失败主要不是“记不住场景”, 而是 ___BOLD_0004___。后续应把“目标类型 → 动作类型”的映射也纳入世界模型 (如 ___CODE_0000___ 对应 ___CODE_0001___ 而非 ___CODE_0002___)。
- ___BOLD_0001___: ___CODE_0000___ 已接入 rule engine, 但本游戏 30 步内未发生显著场景切换或能力翻转, 未触发 stale replan; 世界模型 stats 为空是因为该游戏 driver 未写入观测 (可后续补接入)。

- `__BOLD__0000__`：云端 API 实验显示 kimi-k2.7-code 文本决策 latency 仅 2.2s，远快于旧 deepseek；但将其直接用于本游戏仍需先把动作空间描述清楚。
- `__BOLD__0001__`：`__CODE__0000__` 每步 4 条 verify/decide 消息，墙钟开销 <10%，说明总线本身不是瓶颈；瓶颈在于视觉/云端调用时延（本地 4.8–17s/帧，云端 3.7–38s/帧）。

1.5 4. 本地 VLM 基准 (RTX 5060 Laptop 8GB)

后端：`__CODE__0000__`

参数：`__CODE__0000__`

测试任务：对 3 帧 SSD_00461P01 截图做视觉 struct 提取（JSON schema），记录解析成功率与生成速度。

模型	解析成功	平均墙钟/帧	平均生成 tok/s	备注
Qwen3.5-4B-Q4_K_M	0/3	11.3s	~62	输出为空或 mar
Qwen3.5-9B-Q4_K_M	2/3	16.9s	44.8	成功帧输出完整
<code>__BOLD__0000__</code>	<code>__BOLD__0000__</code>	<code>__BOLD__0000__</code>	<code>__BOLD__0000__</code>	输出紧凑，直接

结论：

- 8GB 显存可同时加载 vision projector + 4-bit 模型；q4_0 KV-cache 量化有效控制显存。
- gemma-4-E4B 在本任务上综合最好（快、稳、解析率高），适合作为本地小模型候选。
- Qwen3.5 为思考模型，必须给足 `__CODE__0000__`（2048）并做更强的输出契约约束；当前 512 token 会被 reasoning 占满。

1.6 5. 云端 API 混合实验

脚本：`__CODE__0000__`（连通性/延迟）、`__CODE__0001__`（视觉 struct 提取）。

1.6.1 5.1 连通性与延迟

1.6.2 5.2 视觉 struct 提取 (JSON schema)

结论：

- mimo-v2.5 是可靠的视觉结构提取器，但成本/延迟高于本地 gemma。

模型	模态	延迟	结果
mimo-v2.5	视觉	17.6s	OK, content 为空（可能为内容过滤或格式问题）
mimo-v2.5	文本	6.7s	OK, 返回”pong”
kimi-k2.7-code	文本	2.2s	OK, 返回”pong”
kimi-k2.6	视觉	3.7s	OK, 返回图片描述
kimi-k2.6	文本	1.4s	OK, 返回”pong”

模型	解析成功	平均延迟	主要问题
mimo-v2.5	3/3	22–38s	输出完整 JSON，但较慢
kimi-k2.6	0/3	8–10s	先输出大量思考过程，1024 token 内未到达 JSON

- kimi-k2.6 需要更强的输出约束（如“先思考，但 __BOLD_0001__”或提高 __CODE_0000__ 到 2048+），否则结构化任务失败。
- 在线控制若预算敏感，可用本地 gemma 做逐帧视觉提取；若精度优先，可用 mimo-v2.5。

1.7 6. Agent 通信与记忆

新增/修复模块：

模块	作用
__CODE_0000__	显式消息总线，__CODE_0000__ 含 OBSERVE/PERCEIVE/DECIDE/VERIFY/C
__CODE_0000__	Observer→StateMapper→DecisionAnalyst→Verifier→Critic 循环，Verifier 可触发 re-
__CODE_0000__	文件型策略记忆，按 __CODE_0000__ 索引成功/失败记录
__CODE_0000__	Critic 角色，给出 diagnosis + correction
__CODE_0000__	注册 __CODE_0000__ / __CODE_0001__ 模式

本轮修复：

- __CODE_0000__ 对 __CODE_0001__ / __CODE_0002__ 为 None 时鲁棒化。
- __CODE_0000__ 与 __CODE_0001__ 兼容 __CODE_0002__ 数据类（之前按 dict 处理崩溃）。

测试：__CODE_0000__、__CODE_0001__、__CODE_0002__、__CODE_0003__ 均通过；合并后 671 passed, 6 failed（与缺失数据集/目录扫描有关）。

1.8 7. 训练准备

- __CODE_0000__ 已转为 7 任务 JSONL（processed-runs 数据）。

- __CODE_0000__ (QLoRA 4bit NF4) 就绪。
- ssh5090 仍不可访问, 本轮未开始训练; 准备好后执行 __CODE_0000__ 同步数据与脚本。

1.9 8. 测试与代码质量

- __CODE_0000__: __BOLD_0001__。
- 失败 1-3: __CODE_0000__ 期望 __CODE_0001__ 目录, 该目录不存在。
- 失败 4-6: __CODE_0000__ 在临时目录测试中把项目根目录的 SSD_00461P01 也扫描进来, 导致计数错误。
- __CODE_0000__: __BOLD_0003__ (本轮修复了 __CODE_0001__ 的 __CODE_0002__ 导入、table_rows nonlocal、未使用导入, 以及新脚本的未使用导入)。

1.10 9. 后续建议 (按优先级)

1. __BOLD_0001__: 为 SSD_00461P01 添加 __CODE_0000__ 动作原语 (点击敌人/塔位/升级按钮) 与对应 profile; 先让 rule 模式能拿分, 再比较 multi-bus/memory 的收益。
2. __BOLD_0003__: 在扩展后的动作空间上, 用 __CODE_0000__ / __CODE_0001__ / __CODE_0002__ 模式跑 10-30 步, 对比 kimi-k2.7-code / mimo-v2.5 的决策质量与成本。
3. __BOLD_0000__: 为视觉结构化任务加 “只输出 JSON” 的系统提示或函数调用/tool 约束, 复测解析率。
4. __BOLD_0001__: 把 gemma-4-E4B 通过 OpenAI 兼容端点接入 __CODE_0000__, 在塔防扩展后的动作空间上做端到端闭环。
5. __BOLD_0000__: 可访问后先用 qwen3.5-4b 做一版微调, 评估本地模型在 failure_recovery / next_probe_action 任务上的提升。
6. __BOLD_0001__: 补齐缺失的 cold-start 数据集或改为 mock; 修复 __CODE_0000__ 扫描根目录的副作用。